

กลุ่มที่18 - Modules, Packages & Crates

จัดทำโดย

1. นายสิปปกร ทองนุ้ม - Concept + Short code Illustration
2. นางสาวอชิรญา ก้อนสุวรรณ - Detail Code + Live Demo
3. นางสาวอนัญญณ์ช์ ขจรศิริผล - Rust vs Other Language
4. นายกฤติน เหลืองระลิก - Exercises + Common Mistake + Challenge

AGENDA

สารบัญ

1

**Concept + Short Code
Illustration**

สรุปแนวคิดหลักและ
โค้ดตัวอย่างสั้น

2

**Detailed Code + Live
Demo**

โค้ดเชิงลึกและสาธิต
การทำงาน

3

**Rust vs Other Language
+ PPL Analysis**

เปรียบเทียบภาษา และ
วิเคราะห์เชิง PPL

4

**Exercises + Common
Mistakes + Challenge**

แบบฝึกหัด, ข้อผิดพลาด
ที่พบบ่อย และคำถาม
ท้าทาย

Introduction

Rust เป็นภาษาการเขียนโปรแกรมที่ให้ความสำคัญกับความปลอดภัย ความถูกต้องของโปรแกรม และการจัดการทรัพยากรอย่างมีประสิทธิภาพ

เมื่อโปรแกรมมีขนาดใหญ่ขึ้น การเขียนโค้ดทั้งหมดไว้ในไฟล์เดียวจะทำให้โปรแกรมอ่านและดูแลได้ยาก Rust จึงมีระบบสำหรับจัดระเบียบโค้ด ได้แก่ Modules, Crates และ Packages รวมถึงคำสั่ง use และระบบ Visibility เช่น pub หัวข้อนี้ช่วยให้ผู้พัฒนาสามารถแบ่งโปรแกรมออกเป็นส่วนย่อย ๆ ตามหน้าที่ ทำให้โค้ดเป็นระเบียบ ลดความซับซ้อน และสามารถนำส่วนต่าง ๆ ของโปรแกรมกลับมาใช้งานได้ง่าย

Concept & Short Code Illustration

PART 1

Important Syntax / Rules

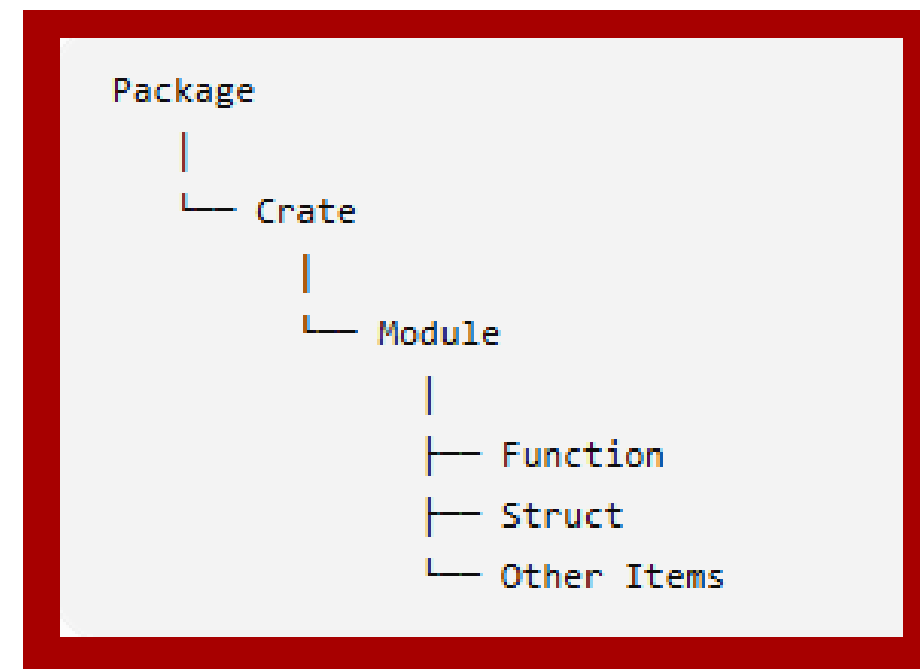
Concept and Background

Modules, Packages & Crates เป็นแนวคิดของ Rust ที่ใช้สำหรับ จัดโครงสร้างและแบ่งโปรแกรมออกเป็นส่วน ๆ เพื่อให้โค้ดเป็นระเบียบและจัดการได้ง่าย โดย Package เป็นหน่วยของโปรเจกต์, Crate เป็นหน่วยที่ Rust ใช้ในการคอมไพล์ และ Module ใช้จัดกลุ่มและควบคุมขอบเขตของโค้ดภายใน Crate

What are Modules, Packages & Crates?

Modules, Packages & Crates เป็นแนวคิดที่ Rust ใช้สำหรับ จัดโครงสร้างโปรเจกต์และจัดระเบียบโค้ด ให้สามารถแบ่งโค้ดออกเป็นส่วน ๆ และควบคุมการเข้าถึงได้อย่างเป็นระบบ

แนวคิดทั้ง 3 มีความสัมพันธ์กัน



ง่าย ๆ

- Package → โครงสร้างของโปรเจกต์
- Crate → หน่วยที่ Rust ใช้ compile
- Module → ใช้จัดระเบียบโค้ดภายใน crate

Package

คือชุดของไฟล์ที่รวมกันเป็นโปรเจกต์ Rust และถูกจัดการโดย Cargo

ตัวอย่าง:

```
my_project/  
├─ Cargo.toml  
└─ src/  
    └─ main.rs
```

- Cargo.toml → เก็บข้อมูลและ configuration ของ package
- src/main.rs → จุดเริ่มต้นของ binary crate

สร้าง package ได้ด้วย:

```
cargo new my_project
```

Key idea:

Package เป็นระดับที่ใช้จัดการโปรเจกต์ และสามารถประกอบด้วย crate ได้

Crate

คือหน่วยของโค้ดที่ Rust compiler สามารถ compile ได้

Crate มี 2 รูปแบบหลัก:

Binary Crate
↓
สร้าง executable
เช่น main.rs

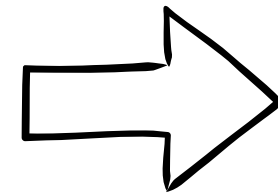
Library Crate
↓
สร้าง library สำหรับนำไปใช้ในโปรเจกต์อื่น
เช่น lib.rs

ตัวอย่าง Binary Crate:

```
fn main() {  
    println!("Hello, Rust!");  
}
```

main.rs ทำหน้าที่เป็น crate root ของ binary crate
ส่วน Library Crate จะมี lib.rs เป็น crate root

Module



Module ใช้สำหรับจัดระเบียบและแบ่งโค้ดภายใน crate

ตัวอย่าง:

```
mod math {  
    pub fn add(a: i32, b: i32) -> i32 {  
        a + b  
    }  
}  
  
fn main() {  
    println!("{}", math::add(2, 3));  
}
```

pub ทำให้ add สามารถถูกเข้าถึงจากภายนอก module ตามกฎ visibility ของ Rust

ตรงนี้:

สร้าง module ชื่อ math

ตรงนี้:

คือการเข้าถึง function add ผ่าน module path

เหตุผลที่ Rust ออกแบบ

เมื่อโปรแกรมมีขนาดใหญ่ขึ้น ถ้าเขียนทุกอย่างไว้ในไฟล์เดียวจะทำให้:

- โค้ดจัดการยาก
- หาส่วนต่าง ๆ ของโปรแกรมได้ยาก
- แบ่งหน้าที่ของโค้ดไม่ชัดเจน
- ควบคุมการเข้าถึงส่วนต่าง ๆ ได้ยาก
- การนำโค้ดกลับมาใช้ซ้ำทำได้ไม่เป็นระบบ

Rust จึงมีแนวคิด Package → Crate → Module เพื่อช่วยจัดโครงสร้างโปรแกรมให้เป็นระบบมากขึ้น

Key Concepts

Short code 3 Concepts ครบคลุมหัวข้อนี้

- Module

- Visibility และ pub

- use และ Path

Key Concepts

Short code 3 Concepts ครบคลุมหัวข้อนี้

- Module

```
mod greeting {  
    fn hello() {  
        println!("Hello!");  
    }  
}
```

greeting คือ Module และ
hello() เป็น Function ที่อยู่
ภายใน Module นั้น

Key Concepts

Short code 3 Concepts ครอบคลุมหัวข้อนี้

- Visibility และ pub

```
mod user {  
    pub struct User {  
        pub name: String,  
    }  
}
```

pub ที่อยู่หน้า struct ทำให้ User สามารถถูกเข้าถึงจากภายนอก Module ได้ ส่วน pub หน้า name ทำให้ Field นี้สามารถถูกเข้าถึงได้เช่นกัน

Key Concepts

Short code 3 Concepts ครบคลุมหัวข้อนี้

- use และ Path

```
mod greeting {  
    pub fn hello() {  
        println!("Hello!");  
    }  
  
    pub fn bye() {  
        println!("Bye Bye!");  
    }  
}  
  
use greeting::{hello, bye};  
  
fn main() {  
    hello();  
    bye();  
}
```

Path ใช้ระบุตำแหน่งของ Item ภายในโครงสร้างของโปรแกรม ส่วน use ช่วยนำ Item ที่ต้องการเข้ามาอยู่ใน Scope เพื่อให้เรียกใช้งานได้สะดวกขึ้น

IMPORTANT SYNTAX

Syntax / Rule	Meaning	Example
mod	ประกาศ Module เพื่อจัดกลุ่มโค้ดและสร้างโครงสร้าง Module Tree	mod front_of_house;
pub	กำหนดให้ Module หรือ Item สามารถเข้าถึงได้จากภายนอกตามกฎ Privacy	pub fn add_to_waitlist() {}
use	นำ Path เข้ามาใน Scope เพื่อให้เรียกใช้งานได้สั้นและสะดวกขึ้น	use crate::front_of_house::hosting;
pub use	นำ Item เข้ามาใน Scope และ Re-export ให้ส่วนอื่นเข้าถึงผ่าน Path ใหม่ได้	pub use crate::front_of_house::hosting;
crate::	ใช้เริ่ม Absolute Path จาก Crate Root ของ Crate ปัจจุบัน	crate::front_of_house::hosting

PART 1 MEMBER 1

Syntax / Rule	Meaning	Example
self::	ใช้อ้างอิงจาก Module ปัจจุบัน	self::hosting
super::	ใช้อ้างอิง Item ที่อยู่ใน Parent Module	super::deliver_order()
::	ใช้ค้นส่วนต่าง ๆ ของ Path ใน Module Tree	front_of_house::hosting
Cargo.toml	ไฟล์ที่อธิบาย Package และข้อมูลที่ Cargo ใช้ในการจัดการ Package	Cargo.toml
src/main.rs	Crate Root ของ Binary Crate ตามโครงสร้างมาตรฐานของ Cargo	fn main() {}
src/lib.rs	Crate Root ของ Library Crate ตามโครงสร้างมาตรฐานของ Cargo	pub mod hosting;

IMPORTANT RULES

Package ต้องมีอย่างน้อย 1
Crate มี Binary Crates ได้
หลายตัว แต่มี Library Crate
ได้ไม่เกิน 1 ตัว

Crate มี 2 ประเภทหลัก คือ
Binary Crate สำหรับสร้าง
โปรแกรมที่รันได้ และ Library
Crate สำหรับสร้าง Library
ให้โปรแกรมอื่นนำไปใช้

Module ใช้จัดโครงสร้างโค้ด
ภายใน Crate โดยใช้ mod
ในการประกาศ Module

Items ใน Module เป็น
Private โดย Default หาก
ต้องการให้ส่วนอื่นเข้าถึงได้
ต้องกำหนดด้วย pub

Path ใช้อ้างอิง Item ใน
Module Tree และ use ช่วย
นำ Path เข้ามาใน Scope เพื่อ
เรียกใช้ได้สะดวกขึ้น

Detail Code & Live Demo

PART 2

Runnable Code Example

Example 1: Module, Visibility และ pub

```
mod calculator {  
    pub fn add(a: i32, b: i32) -> i32 {  
        a + b  
    }  
  
    fn secret_operation(a: i32, b: i32) -> i32 {  
        a * b  
    }  
}  
  
fn main() {  
    let result = calculator::add(10, 20);  
  
    println!("Result = {}", result);  
}
```

ตัวอย่างที่ 1 นี้
แสดงวิธีสร้าง Module
และกำหนดว่า Function
ใดสามารถถูกเรียกใช้งาน
จากภายนอก Module ได้

Example 1: Module, Visibility และ pub

```
mod calculator {  
    pub fn add(a: i32, b: i32) -> i32 {  
        a + b  
    }  
  
    fn secret_operation(a: i32, b: i32) -> i32 {  
        a * b  
    }  
}  
  
fn main() {  
    let result = calculator::add(10, 20);  
  
    println!("Result = {}", result);  
}
```

Function add
ถูกประกาศด้วย pub

pub ย่อมาจาก public
และทำให้ Function นี้
สามารถถูกเรียกใช้งาน
จากภายนอก Module ได้

Example 1: Module, Visibility และ pub

```
mod calculator {  
    pub fn add(a: i32, b: i32) -> i32 {  
        a + b  
    }  
  
    fn secret_operation(a: i32, b: i32) -> i32 {  
        a * b  
    }  
}  
  
fn main() {  
    let result = calculator::add(10, 20);  
  
    println!("Result = {}", result);  
}
```

Function
secret_operation
ไม่ได้ใช้ pub

ดังนั้น Function นี้
จึงเป็น private
สามารถใช้งานได้ภายใน
Module calculator
เท่านั้น

Example 1: Module, Visibility และ pub

```
mod calculator {  
    pub fn add(a: i32, b: i32) -> i32 {  
        a + b  
    }  
  
    fn secret_operation(a: i32, b: i32) -> i32 {  
        a * b  
    }  
}  
  
fn main() {  
    let result = calculator::add(10, 20);  
    println!("Result = {}", result);  
}
```

การเรียกใช้งาน Function

โดย :: ใช้สำหรับ
เข้าถึงสิ่งที่อยู่ภายใน
Module ผ่าน Module Path

Example 2: การใช้ use กับหลาย Module

จากตัวอย่าง Project
modules_demo แบ่ง
Code ออกเป็น 3 ไฟล์

- main.rs → โปรแกรมหลัก
- calculator.rs → Function สำหรับการคำนวณ
- utils.rs → Function สำหรับงานทั่วไป

Example 2: การใช้ use กับหลาย Module

```
pub fn add(a: i32, b: i32) -> i32 {  
    a + b  
}  
  
pub fn subtract(a: i32, b: i32) -> i32 {  
    a - b  
}  
  
pub fn multiply(a: i32, b: i32) -> i32 {  
    a * b  
}  
  
pub fn divide(a: i32, b: i32) -> f64 {  
    a as f64 / b as f64  
}
```

- calculator.rs → Function สำหรับการคำนวณ

เก็บ Function ที่
เกี่ยวข้องกับการคำนวณ

Example 2: การใช้ use กับหลาย Module

```
pub fn print_title(title: &str) {  
    println!("=====");  
    println!("{}", title);  
    println!("=====");  
}
```

- `utils.rs` → Function สำหรับงานทั่วไป

เก็บ Function สำหรับงานทั่วไป ในตัวอย่างจะเป็นแพทเทิร์นหัวข้อ

Example 2:

การใช้ use กับหลาย Module

```
mod calculator;
mod utils;

use calculator::{add, subtract, multiply, divide};
use utils::print_title;

fn main() {
    print_title("Rust Modules Demo");

    let a = 20;
    let b = 10;

    println!("Addition: {}", add(a, b));
    println!("Subtraction: {}", subtract(a, b));
    println!("Multiplication: {}", multiply(a, b));
    println!("Division: {}", divide(a, b));
}
```

- main.rs → โปรแกรมหลัก

Example 2: การใช้ use กับหลาย Module

```
mod calculator;
mod utils;

use calculator::{add, subtract, multiply, divide};
use utils::print_title;

fn main() {
    print_title("Rust Modules Demo");

    let a = 20;
    let b = 10;

    println!("Addition: {}", add(a, b));
    println!("Subtraction: {}", subtract(a, b));
    println!("Multiplication: {}", multiply(a, b));
    println!("Division: {}", divide(a, b));
}
```

- main.rs → โปรแกรมหลัก

ใช้เพื่อบอก Rust ว่า
โปรแกรมนี้นี้
Module ที่ชื่อว่า
calculator และ utils

Example 2: การใช้ use กับหลาย Module

```
mod calculator;
mod utils;

use calculator::{add, subtract, multiply, divide};
use utils::print_title;

fn main() {
    print_title("Rust Modules Demo");

    let a = 20;
    let b = 10;

    println!("Addition: {}", add(a, b));
    println!("Subtraction: {}", subtract(a, b));
    println!("Multiplication: {}", multiply(a, b));
    println!("Division: {}", divide(a, b));
}
```

- main.rs → โปรแกรมหลัก

ใช้คำสั่ง use เพื่อนำ
Function ที่ต้องการมาใช้
งานใน Scope ปัจจุบัน

Example 2: การใช้ use กับหลาย Module

```
mod calculator;
mod utils;

use calculator::{add, subtract, multiply, divide};
use utils::print_title;

fn main() {
    print_title("Rust Modules Demo");

    let a = 20;
    let b = 10;

    println!("Addition: {}", add(a, b));
    println!("Subtraction: {}", subtract(a, b));
    println!("Multiplication: {}", multiply(a, b));
    println!("Division: {}", divide(a, b));
}
```

- main.rs → โปรแกรมหลัก

เมื่อใช้ use แล้ว สามารถ
เขียนสั้นลงได้ดังตัวอย่าง
แต่หากไม่ใช้ use เราจะ
ต้องเรียก Function ด้วย
Full Path

`calculator::add(20, 10);`

Example 2:

การใช้ use กับหลาย Module

```
mod calculator;
mod utils;

use calculator::{add, subtract, multiply, divide};
use utils::print_title;

fn main() {
    print_title("Rust Modules Demo");

    let a = 20;
    let b = 10;

    println!("Addition: {}", add(a, b));
    println!("Subtraction: {}", subtract(a, b));
    println!("Multiplication: {}", multiply(a, b));
    println!("Division: {}", divide(a, b));
}
```

ผลลัพธ์ที่ได้

```
=====
Rust Modules Demo
=====

Addition: 30
Subtraction: 10
Multiplication: 200
Division: 2
```

Example 3: การใช้ Module ใน Rust

จากตัวอย่าง Project
module_demo2 แบ่ง
Code ออกเป็น 3 ไฟล์

- main.rs → โปรแกรมหลัก
- student.rs → Function สำหรับการคำนวณเกรด
- utils.rs → Function ช่วยจัดรูปแบบแสดงผล

Example 3: การใช้ Module ใน Rust

เก็บ Function ที่เกี่ยวข้องกับ
กับการคำนวณเกรด

- student.rs → Function สำหรับการคำนวณเกรด

```
pub fn calculate_grade(score: f32) -> &'static str {  
    if score >= 80.0 {  
        "A"  
    } else if score >= 70.0 {  
        "B"  
    } else if score >= 60.0 {  
        "C"  
    } else if score >= 50.0 {  
        "D"  
    } else {  
        "F"  
    }  
}
```

```
fn get_message(score: f32) -> &'static str {  
    if score >= 80.0 {  
        "Excellent!"  
    } else if score >= 50.0 {  
        "Passed"  
    } else {  
        "Failed"  
    }  
}  
  
pub fn display_result(name: &str, score: f32) {  
    println!("Student: {}", name);  
    println!("Score: {}", score);  
    println!("Grade: {}", calculate_grade(score));  
    println!("Status: {}", get_message(score));  
}
```

Example 3: การใช้ Module ใน Rust

```
pub fn print_header(title: &str) {  
    println!();  
    println!("=====");  
    println!("{}", title);  
    println!("=====");  
}  
  
pub fn print_sep() {  
    println!("-----");  
}
```

- utils.rs → Function ช่วยจัดรูปแบบแสดงผล

เก็บ Function ช่วยจัดรูปแบบการแสดงผล

Example 3: การใช้ Module ใน Rust

```
mod student;
mod utils;

use student::{calculate_grade, display_result};
use utils::{print_header, print_sep};

fn main() {
    print_header("Student Grade Demo");

    let achiraya_score = 85.0;
    let somchai_score = 45.0;

    display_result("Achiraya", achiraya_score);

    print_sep();

    display_result("Somchai", somchai_score);
}
```

- main.rs → โปรแกรมหลัก

โปรแกรมทำงานโดยรับค่า
คะแนนนักเรียนแล้วเรียกใช้
ฟังก์ชัน display_result จาก
Module student คำนวณ
และแสดงข้อมูลนักเรียน

Example 3: การใช้ Module ใน Rust

```
mod student;
mod utils;

use student::{calculate_grade, display_result};
use utils::{print_header, print_sep};

fn main() {
    print_header("Student Grade Demo");

    let achiraya_score = 85.0;
    let somchai_score = 45.0;

    display_result("Achiraya", achiraya_score);

    print_sep();

    display_result("Somchai", somchai_score);
}
```

ผลลัพธ์ที่ได้

```
=====
Student Grade Demo
=====
Student: Achiraya
Score: 85
Grade: A
Status: Excellent!
-----
Student: Somchai
Score: 45
Grade: F
Status: Failed
```

Rust vs Other Language

PART 3

PPL Perspective

RUST VS OTHER LANGUAGE

ASPECT	RUST	JAVA	C++	PYTHON
Syntax	mod, pub, use	package, import	module, export, import	.py, import
Semantics	Package → Crate → Module	Package → Class/Interface	Module → Exported Declarations	Package → Module
Type System	Module: Function, Struct, Enum	Package: Class, Interface	Module: Function, Class, Type	Module: Function, Class
Memory	Module System เน้นจัดโครงสร้าง Code ไม่ได้จัดการ Memory โดยตรง	Package เน้นจัดกลุ่ม Code ไม่ได้จัดการ Memory โดยตรง	Module เน้นแบ่งและเชื่อม Code ไม่ได้จัดการ Memory โดยตรง	Module/Package เน้นจัดกลุ่ม Code ไม่ได้จัดการ Memory โดยตรง
Visibility	Private default, pub	public, private, protected	export + Access Specifiers	Convention เช่น _name

ANALYSIS

01 – STRUCTURE & MODULARITY

- Rust: Package → Crate → Module
- Java: Package → Class / Interface
- C++: Module → Exported Declarations
- Python: Package → Module

02 – SCOPE & VISIBILITY

- Rust: Private Default + pub
- Java: Access Modifiers
- C++: export + Access Specifiers
- Python: Convention `_name`

03 – SYNTAX & ORGANIZATION

- Rust: `mod` / `pub` / `use`
- Java: `package` / `import`
- C++: `module` / `export` / `import`
- Python: `.py` / `import`

04 – KEY DIFFERENCE

Rust เน้นการจัดโครงสร้างและควบคุม Public / Private Interface ผ่าน Module System อย่างชัดเจน ต่างจากภาษาอื่นที่ใช้กลไกตามโครงสร้างของภาษา

PPL PERSPECTIVE

Syntax

- mod** สร้าง Module
- pub** เปิดให้ภายนอกเข้าถึง
- use** นำ Path เข้ามาใน Scope
- crate** อ้างถึง Crate Root ของ Crate ปัจจุบัน
- super** อ้างถึง Parent Module
- self** อ้างถึง Module ปัจจุบัน
- ::** ใช้แบ่งระดับของ Path

Semantics

Package

Crate

Module

Item

Type System

Module สามารถเก็บ Item
เช่น Function, Struct,
Enum

Module System ไม่ได้
กำหนด Type System
โดยตรง

ช่วยจัดกลุ่มและกำหนด
ขอบเขตของ Item

PPL PERSPECTIVE

Memory / Resource

Package, Crate และ Module ไม่ได้จัดการ Memory โดยตรง

เน้นจัดการ:

Organization

Scope

Privacy

Public Interface

Private Interface

Abstraction

Abstraction

Module รวม Functionality เรียกผ่าน Public Interface

Scope

กำหนดขอบเขตที่ Item สามารถมองเห็นและเรียกใช้งานได้

Binding

food::order()

Encapsulation

pub เปิด Public Interface

หากไม่มี pub ซ่อนเป็น Private

Paradigm

สนับสนุน Modular Programming

แบ่งโปรแกรมขนาดใหญ่ออกเป็นส่วนย่อยที่มีหน้าที่ชัดเจน

Why RUST?

- แบ่ง Code เป็นส่วนย่อย
- จัดกลุ่ม Functionality ที่เกี่ยวข้อง
- กำหนด Public / Private Interface
- จัดการ Scope และ Privacy
- จัดโครงสร้างโปรแกรมขนาดใหญ่ได้ชัดเจน

Mistake & Exercises & Challenge

PART 4

Learn • Practice

COMMON MISTAKE

Mistake 1 : **ลืม pub**

✗ Incorrect

```
mod bank {  
    fn deposit(a: u32, b: u32) -> u32 {  
        a + b  
    }  
}  
  
fn main() {  
    println!("{}", bank::deposit(100, 50));  
}
```

Why?

Rust ทุก item เป็น private by default ต้องใส่ pub เพื่อเปิดให้ภายนอกใช้ → ตรวจสอบ compile (E0603)

COMMON MISTAKE

Mistake 1 : **lưu pub**



Correct

```
mod bank {  
    pub fn deposit(a: u32, b: u32) -> u32 {  
        a + b  
    }  
}  
  
fn main() {  
    println!("{}", bank::deposit(100, 50));  
}
```

COMMON MISTAKE

Mistake 2 : pub struct แต่ field ยัง private

✗ Incorrect

```
mod bank {  
    pub struct Account {  
        pub owner: String,  
        balance: u32,  
    }  
}  
  
fn main() {  
    let a = bank::Account {  
        owner: String::from("A"),  
        balance: 100, // ✗ E0451  
    };  
}
```

Why?

pub ควบคุมทีละระดับ: struct เปิด
แล้ว แต่ field ยังปิด → สร้างด้วย
literal ไม่ได้ (E0451) ให้ทำ pub fn
new + getter เพื่อรักษา invariant

COMMON MISTAKE

Mistake 2 : **pub struct field ၌ private**



Correct

```
mod bank {  
    pub struct Account {  
        pub owner: String,  
        balance: u32,  
    }  
    impl Account {  
        pub fn new(o: &str, b: u32) -> Account {  
            Account { owner: String::from(o), balance: b }  
        }  
        pub fn balance(&self) -> u32 { self.balance }  
    }  
}  
fn main() {  
    let a = bank::Account::new("A", 100);  
    println!("{}", a.balance());  
}
```

COMMON MISTAKE

Mistake 3 : สร้างไฟล์ .rs แล้วคิดว่า Rust จะรู้จักเอง (ลืม mod)

โครงสร้างโปรเจกต์

```
my_app/  
├─ Cargo.toml  
└─ src/  
    ├─ main.rs  
    └─ utils.rs
```

✗ Incorrect

```
use crate::utils::greet;  
  
fn main() {  
    greet();  
}
```

Why?

ใน Rust module tree ถูกสร้างจากการประกาศ mod ไม่ใช่จากโครงสร้างไฟล์ คำสั่ง mod utils; บอก compiler ให้ไปอ่านโค้ดจาก utils.rs (หรือ utils/mod.rs) มาใส่ไว้เป็น module ชื่อ utils ส่วน use เป็นเพียงการตั้งชื่อลัดให้ path ที่มีอยู่แล้ว ไม่ได้สร้าง module ขึ้นมา (mod = สร้าง/ประกาศ, use = นำเข้าชื่อ)

COMMON MISTAKE

Mistake 3 : สร้างไฟล์ .rs แล้วคิดว่า Rust จะรู้จักเอง (ลืม mod)



Correct

```
mod utils;  
use utils::greet;  
  
fn main() {  
    greet();  
}
```

COMMON MISTAKE

Mistake 4 : คิดว่า module ลูก "เห็น" ชื่อของ module แม่โดยอัตโนมัติ

 **Incorrect**

```
fn helper() -> u32 {
    42
}

mod inner {
    pub fn run() -> u32 {
        helper()
    }
}

fn main() {
    println!("{}", inner::run());
}
```

Why?

Rust ใช้ module เป็นขอบเขตของ namespace ชื่อไม่ไหลข้ามขอบเขตโดยอัตโนมัติ (ผู้อ่านโค้ดจึงเห็นชัดว่าแต่ละ module ฟังทำอะไร) สังเกตว่า helper เป็น private แต่ module ลูกยังเรียกได้ เพราะกฎ privacy คือ "item เป็น private = มองเห็นได้เฉพาะใน module ที่ประกาศมัน และ module ลูกหลานของมัน"

 **Correct**

```
fn helper() -> u32 {
    42
}

mod inner {
    pub fn run() -> u32 {
        super::helper()
    }
}

fn main() {
    println!("{}", inner::run());
}
```

Exercise 1 – สร้าง Module ซ้อนกัน (Basic)

Problem

เขียนโปรแกรมไฟล์เดียวที่มี module shapes ซึ่งมี module ย่อย 2 ตัว คือ circle และ rectangle โดยแต่ละตัวมีฟังก์ชัน area ที่คำนวณพื้นที่ แล้วเรียกใช้จาก main ด้วย use โดยต้องตั้งชื่อลัด (as) ให้ rectangle::area เพื่อไม่ให้ชื่อชนกัน

ผลลัพธ์ที่ต้องได้:

```
Circle (r=2): 12.57  
Rectangle (3x4): 12.00
```

Hint 

- module ย่อยที่อยู่ใน module shapes ต้องเป็น pub mod และฟังก์ชันข้างในต้องเป็น pub fn
- ค่า π ใช้ std::f64::consts::PI
- ทศนิยม 2 ตำแหน่งใช้ {:.2}
- การตั้งชื่อลัด: use shapes::rectangle::area as rect_area;

Exercise 1 – สร้าง Module ชื่อนกน (Basic)

Solution



```
mod shapes {  
    pub mod circle {  
        pub fn area(r: f64) -> f64 {  
            std::f64::consts::PI * r * r  
        }  
    }  
  
    pub mod rectangle {  
        pub fn area(w: f64, h: f64) -> f64 {  
            w * h  
        }  
    }  
}  
  
use shapes::circle;  
use shapes::rectangle::area as rect_area;  
  
fn main() {  
    println!("Circle (r=2): {:.2}", circle::area(2.0));  
    println!("Rectangle (3x4): {:.2}", rect_area(3.0, 4.0));  
}
```

Exercise 2 — สร้าง Package ที่มี Library Crate + Binary Crate (Normal)

Problem

ใช้ Cargo สร้าง package ชื่อ `thai_greeter` ที่ประกอบด้วย

1. library crate (`src/lib.rs`) ที่มี `pub mod greeting`;
2. ไฟล์ `src/greeting.rs` ที่มีฟังก์ชัน `private decorate` และฟังก์ชัน `public hello(name: &str) -> String`, โดย `hello` ต้องเรียกใช้ `decorate`
3. binary crate (`src/main.rs`) ที่เรียก `hello("Rust")` จาก library แล้วพิมพ์ผล

จากนั้นตอบคำถาม 2 ข้อ

- ก. package นี้มีกี่ crate และเป็นชนิดอะไรบ้าง
- ข. ถ้าลองให้ `main.rs` เรียก `thai_greeter::greeting::decorate(...)` ตรง ๆ จะเกิดอะไรขึ้น เพราะอะไร

ผลลัพธ์ของ `cargo run`:

```
*** สวัสดี Rust ***
```

Hint 

- สร้างด้วย `cargo new thai_greeter --lib` จะได้ `src/lib.rs` ให้ แล้วสร้าง `src/main.rs` เอง Cargo จะถือว่า มี binary crate ให้อัตโนมัติ
- `main.rs` อ้างถึง library ด้วยชื่อ package (ขีดกลางแปลงเป็นขีดล่าง) เช่น use `thai_greeter::greeting::hello`;
- `decorate` ไม่ต้องใส่ `pub`

Exercise 2 — สร้าง Package ที่มี Library Crate + Binary Crate (Normal)

Solution

Cargo.toml

```
[package]
name = "thai_greeter"
version = "0.1.0"
edition = "2021"
```

src/lib.rs

```
pub mod greeting;
```

src/main.rs

```
use thai_greeter::greeting::hello;

fn main() {
    println!("{}", hello("Rust"));
}
```

src/greeting.rs

```
fn decorate(text: &str) -> String {
    format!("*** {} ***", text)
}

pub fn hello(name: &str) -> String {
    decorate(&format!("สวัสดี {}", name))
}
```

Key takeaways

สรุปสิ่งที่ควรจำ

Package Crate
Module
เป็นโครงสร้างหลักที่
Rust ใช้จัดระเบียบโปร
เจกต์และแบ่งโค้ดเป็น
ส่วน ๆ

mod, pub, use และ
Path
mod ใช้ประกาศ
Module, pub เปิดการ
เข้าถึง และ use ช่วยนำ
Path เข้ามาใน Scope

Private by Default
Item ใน Module เป็น
Private โดยค่าเริ่มต้น
ช่วยกำหนด Public
Interface และซ่อนราย
ละเอียดภายใน

เชื่อมโยงกับ PPL:
Modularity, Scope,
Privacy & Encapsulation
Module System ช่วยจัด
โครงสร้าง กำหนดขอบเขต
การมองเห็น และแยก Public
Interface ออกจาก
Implementation Details

References

- The Rust Programming Language — Managing Growing Projects with Packages, Crates, and Modules
<https://doc.rust-lang.org/book/ch07-00-managing-growing-projects-with-packages-crates-and-modules.html>
- Rust by Example — Modules
<https://doc.rust-lang.org/rust-by-example/mod.html>
- Rust Module (With Example)
<https://www.programiz.com/rust/module>
- C++ Reference — Modules (C++20)
<https://en.cppreference.com/w/cpp/language/modules>
- Java Tutorials — Creating and Using Packages
<https://docs.oracle.com/javase/tutorial/java/package/index.html>
- Python Documentation — Modules
<https://docs.python.org/3/tutorial/modules.html>

Questions & Answers

กลุ่มที่18 - Modules, Packages & Crates